

4.0 Introduction [Curve Fitting and Interpolation]

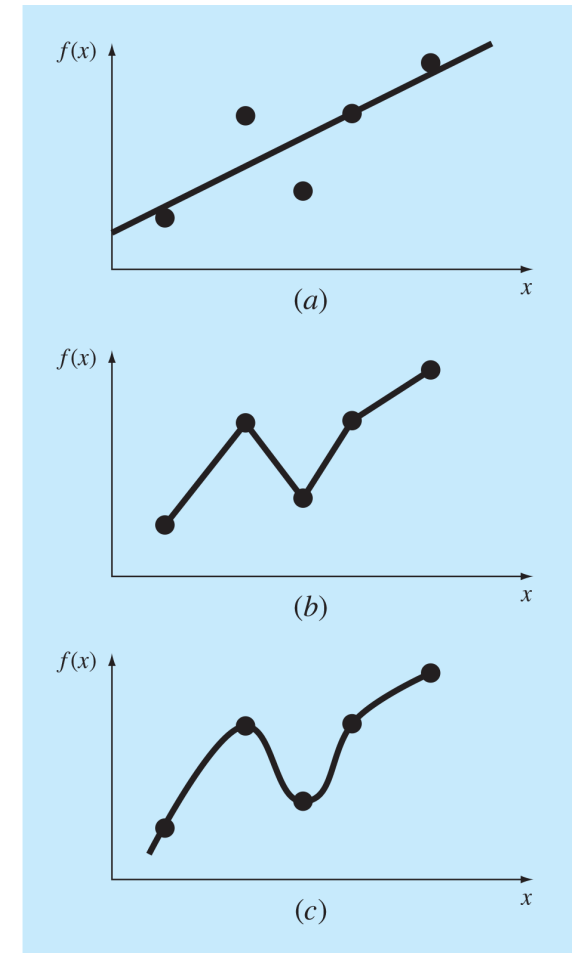
Section 17.1, 17.2 and PT5.1 in Chapra 8th edition

We measure data as discrete values along a continuum, but may require a mathematical representation of the data (curve fit), or require estimates at points between discrete values (interpolation).

Two general approaches:

1. Curve fitting – Derive a single curve that represents the general trend of the data. This is useful for noisy data, or where we require a smooth or continuous mathematical function to describe the data. Considerations include the shape of the function that is fit to the data (e.g. linear, polynomial, exponential etc).

2. Interpolation – Fitting of a curve or series of curves that pass through each data point to enable estimation of intermediate values (between data points).



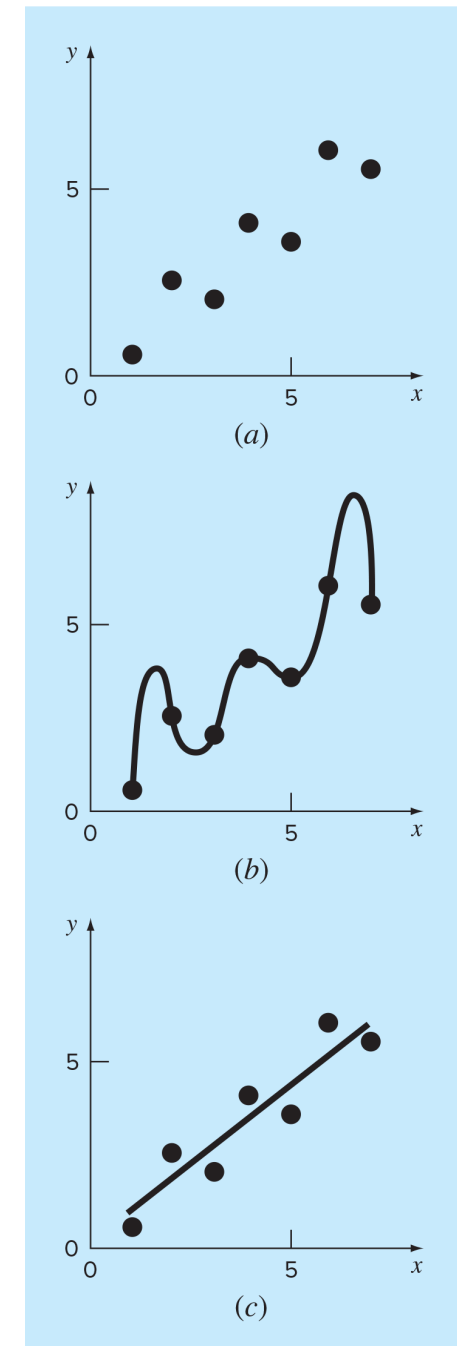
Three approaches to fit a “best” curve through five data points. (a) Least-squares regression, (b) linear interpolation, and (c) curvilinear interpolation.

4.1 Curve Fitting [Least-Squares Regression]

Consider a set of data (top right). We may want a function that passes through all data points (if that is a valid representation of the response between the data points), or we may want to see the general trend of the data.

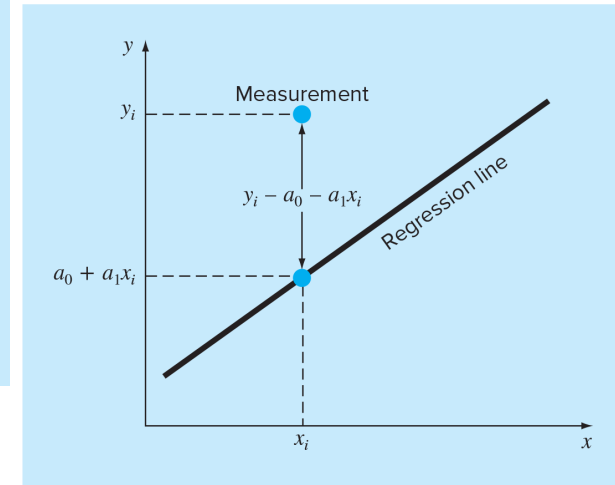
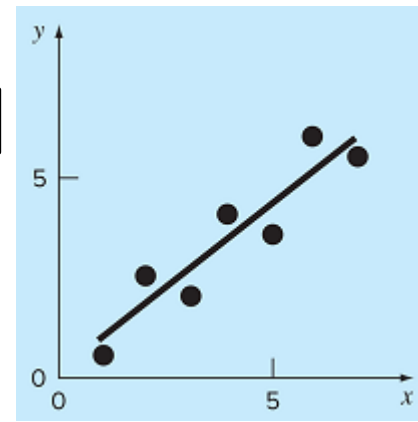
In engineering, it is critical to have consistent and reproducible methods. [documentation is important!]

Let's look at how we can fit a trend line to data using the method of Least-Squares Regression. This method seeks to minimize the residual or error between the data points and the corresponding points on the curve (trend line).



4.1 Curve Fitting [Least Squares Regression]

Consider the equation of a straight line (and corresponding data points):



A common strategy to achieve a best fit is to minimize the sum of squares of the residuals (S_r).

4.1 Curve Fitting [Least-Squares Regression]

Least-Squared Fit of a Straight Line

Differentiate the Sum of Residuals with respect to the coefficients (all summations 1 to n, for n data points):

$$\frac{\partial S_r}{\partial a_0} = -2 \sum (y_i - a_0 - a_1 x_i)$$

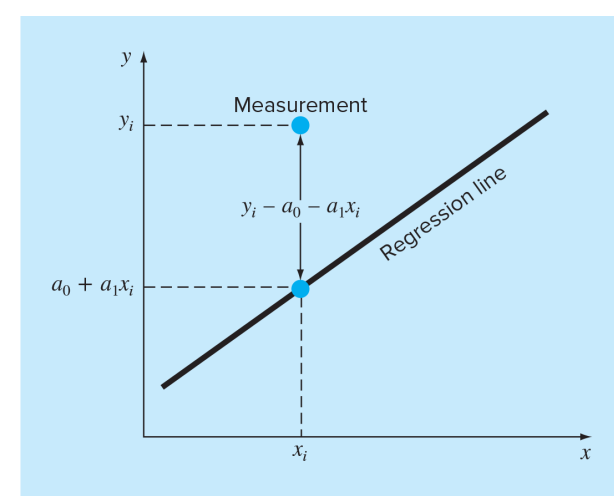
$$\frac{\partial S_r}{\partial a_1} = -2 \sum [(y_i - a_0 - a_1 x_i) x_i]$$

Setting the derivatives equal to zero will minimize S_r , all summations from 1 to n:

$$0 = \sum y_i - \sum a_0 - \sum a_1 x_i$$

$$0 = \sum y_i x_i - \sum a_0 x_i - \sum a_1 x_i^2$$

$$\sum a_0 = n a_0$$



Give the normal equations:

$$n a_0 + \left(\sum x_i \right) a_1 = \sum y_i$$

$$\left(\sum x_i \right) a_0 + \left(\sum x_i^2 \right) a_1 = \sum x_i y_i$$

We can solve for the coefficients:

$$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - \left(\sum x_i \right)^2}$$

$$a_0 = \bar{y} - a_1 \bar{x}$$

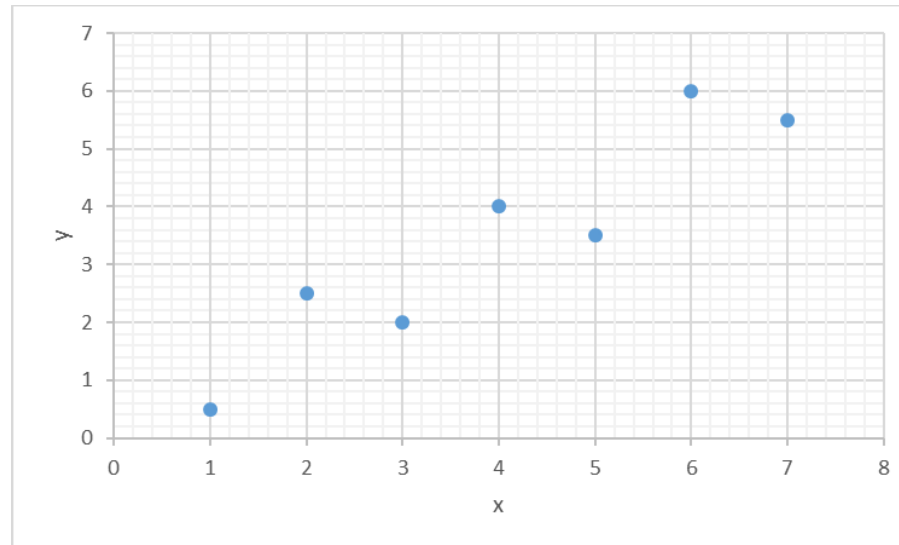
where $\bar{y}$ and $\bar{x}$ are the means of y and x

4.1 Curve Fitting [Least-Squares Regression]

Least-Squared Fit of a Straight Line, Example 4.1

Fit a straight line to the following data set:

<u>x_i</u>	<u>y_i</u>
1	0.5
2	2.5
3	2
4	4
5	3.5
6	6
7	5.5



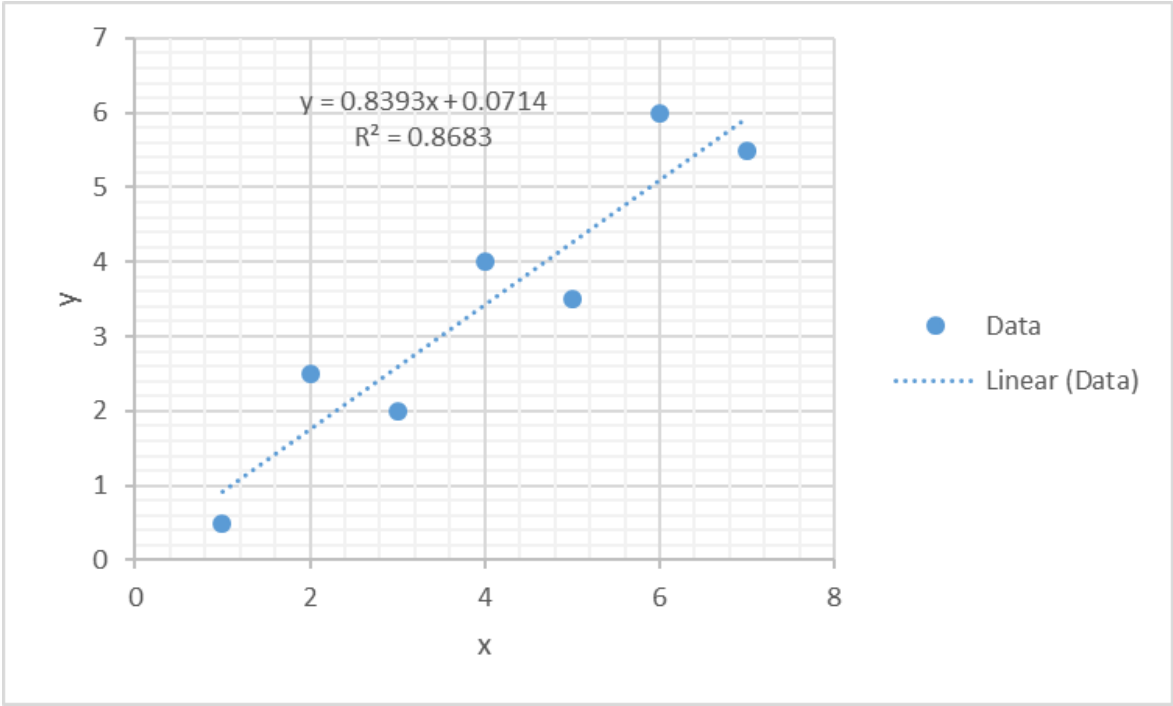
4.1 Curve Fitting [Least-Squares Regression]

Least-Squared Fit of a Straight Line, Example 4.1

Fit a straight line to the following data set:

	<u>xi</u>	<u>yi</u>	<u>xi * yi</u>	<u>xi^2</u>	
	1.0	0.5	0.5	1.0	
	2.0	2.5	5.0	4.0	
	3.0	2.0	6.0	9.0	
	4.0	4.0	16.0	16.0	
	5.0	3.5	17.5	25.0	
	6.0	6.0	36.0	36.0	
	7.0	5.5	38.5	49.0	
sums ->	28.0	24.0	119.5	140.0	
n	xbar	ybar			
	7	4	3.428571		
	a1	0.839286			$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$
	a0	0.071429			
					$a_0 = \bar{y} - a_1 \bar{x}$

Using the trend line function in Excel:



4.1 Curve Fitting [Least-Squares Regression]

How good is the fit?
aka 'Goodness of fit'

The sum of squares of the residuals (S_r) is a quantification of the error (between the measured and predicted y values).

Coefficient of Determination (r^2)

And

Correlation Coefficient (r)

$$r^2 = \frac{S_t - S_r}{S_t}$$

The value of r^2 describes how much of the uncertainty is explained by the model.

$r^2 = 1$ is good (100%)

$r^2 = 0$ is not good

S_t is the total sum of squares around the mean value.

$$S_r = \sum_{i=1}^n (y_i - a_0 - a_1 x_i)^2$$

* An alternate formula for the correlation coefficient (r – not r^2) uses the sums, however there is an error in Equation 17.11 in the text. The correct equation is:

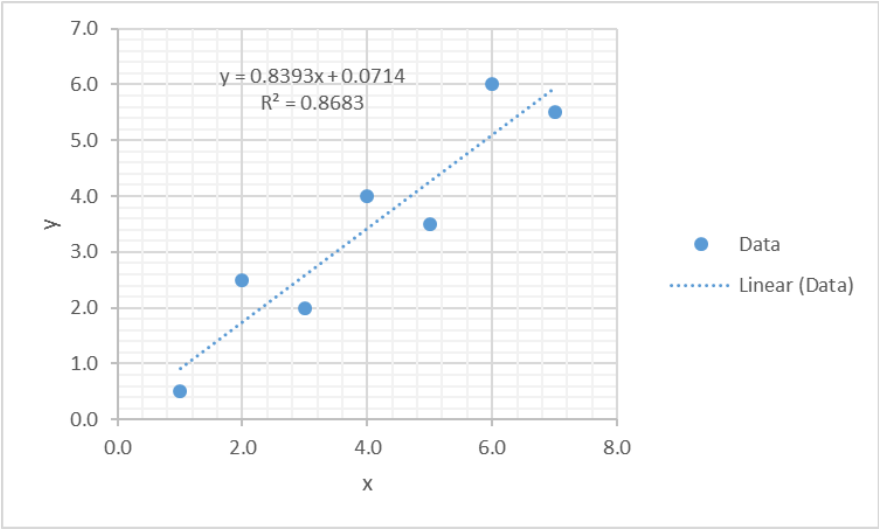
$$r = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{\sqrt{n \sum x_i^2 - (\sum x)^2} \sqrt{n \sum y_i^2 - (\sum y)^2}}$$

Error in 17.11

$$r = \frac{n \sum x_i y_i - (\sum x_i)(\sum y_i)}{\sqrt{n \sum x_i^2 - (\sum x_i)^2} \sqrt{n \sum y_i^2 - (\sum y_i)^2}}$$

4.1 Curve Fitting [Least-Squares Regression]

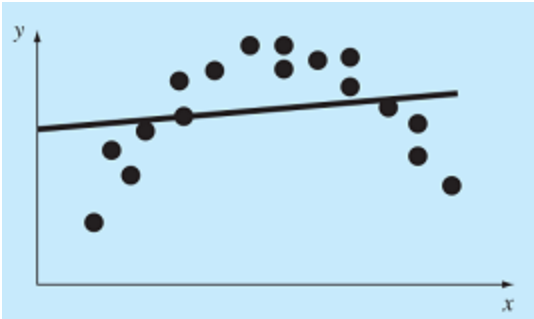
How good is the fit?
aka 'Goodness of fit'



$r^2 = 1$ is a so-called perfect fit ($S_r = S_t$)
Meaning the fit line explains 100% of the variability of the data
 $r^2 = 0$ is a poor fit

In our case, $r^2 = 0.868$, meaning that 86.8% of the uncertainty is explained by the linear model.

*** r^2 close to 1.0 does not always imply a good fit. Important to plot and check results visually.



					Goodness of Fit		
					Sr	St	
					<u>(yi-ao-a1xi)^2</u>	<u>(yi-ybar)^2</u>	
	xi	yi	xi * yi	xi^2			
	1.0	0.5	0.5	1.0		0.1687	8.5765
	2.0	2.5	5.0	4.0		0.5625	0.8622
	3.0	2.0	6.0	9.0		0.3473	2.0408
	4.0	4.0	16.0	16.0		0.3265	0.3265
	5.0	3.5	17.5	25.0		0.5896	0.0051
	6.0	6.0	36.0	36.0		0.7972	6.6122
	7.0	5.5	38.5	49.0		0.1993	4.2908
sums ->	28.0	24.0	119.5	140.0		2.9911	22.7143
n	xbar	ybar					
7	4	3.428571					
	a1	0.839286	$a_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$				
	a0	0.071429					
			$a_0 = \bar{y} - a_1 \bar{x}$				
	r^2	0.868318					
	r	0.931836	$r^2 = \frac{S_t - S_r}{S_t}$				

4.2 Curve Fitting [Least-Squares Regression for non-linear equations]

Linear regression (as the name suggests) requires that the relationship between the dependent and independent variables be linear.

Let's look at how we can apply our method to non-linear equations.

If we can re-write our non-linear equation in a linear form, then we can apply linear regression.

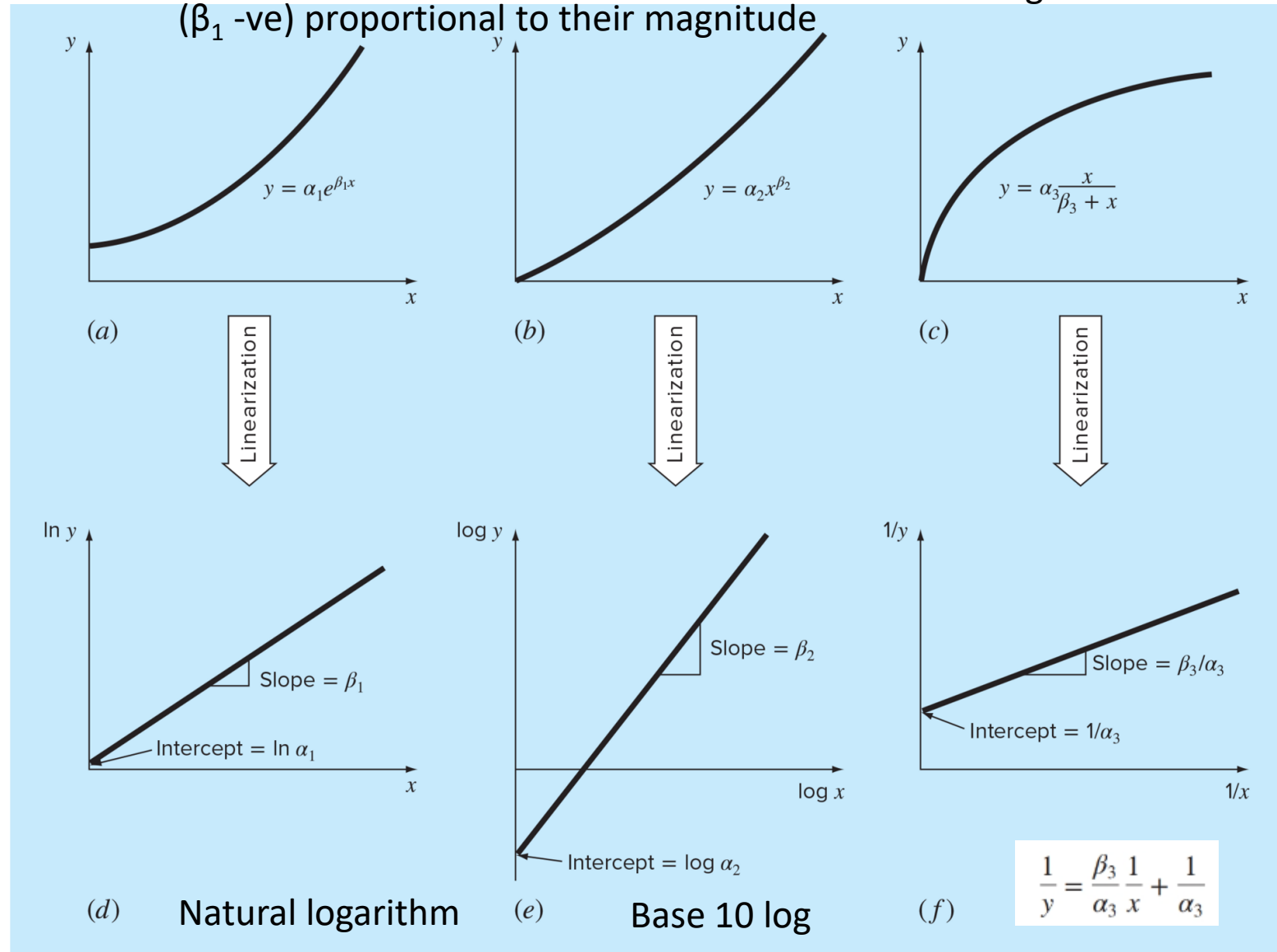
e.g. $y = \alpha_1 e^{\beta_1 x}$

4.2 Curve Fitting [Least-Squares Regression for non-linear equations]

Linearizing non-linear equations so that we can apply linear regression.

These equations are common in engineering applications (population growth or radioactive decay)

e.g. quantities that increase (β_1 +ve) or decrease (β_1 -ve) proportional to their magnitude e.g. saturation

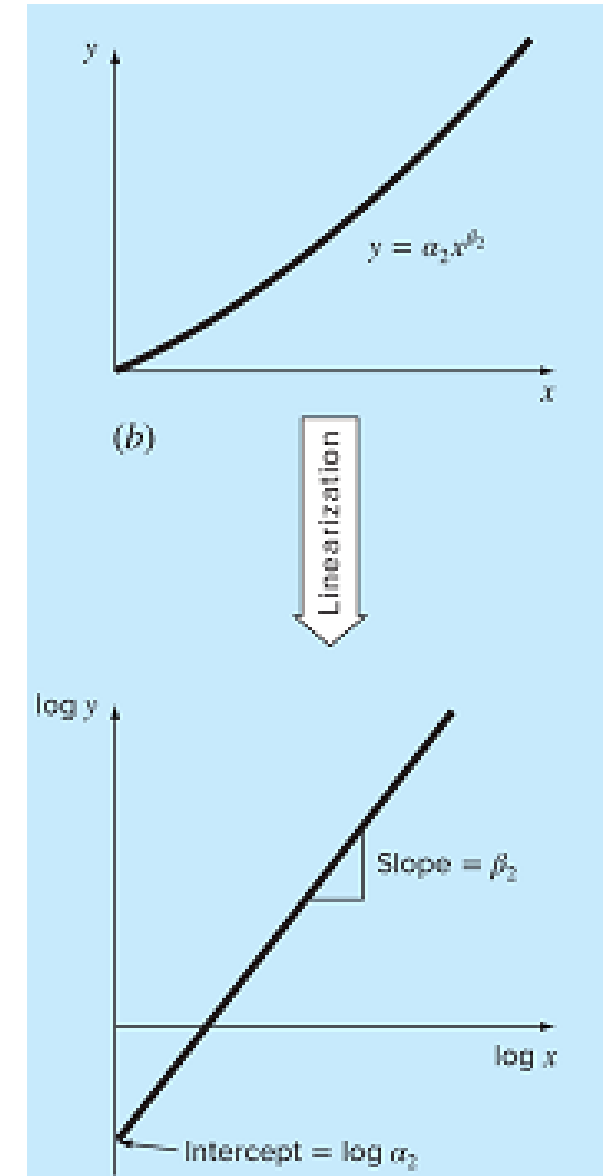
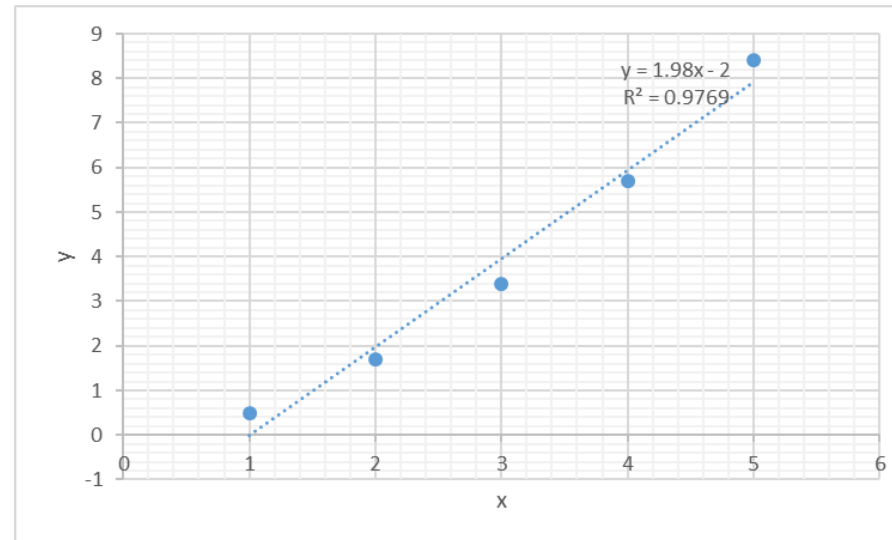


4.2 Curve Fitting [Least-Squares Regression for non-linear equations]

Example 4.2 – non-linear equation

Consider the data below. A straight-line fit to the data shows a good r^2 value, but clearly does not fit the shape of the curve.

<u>xi</u>	<u>yi</u>
1	0.5
2	1.7
3	3.4
4	5.7
5	8.4

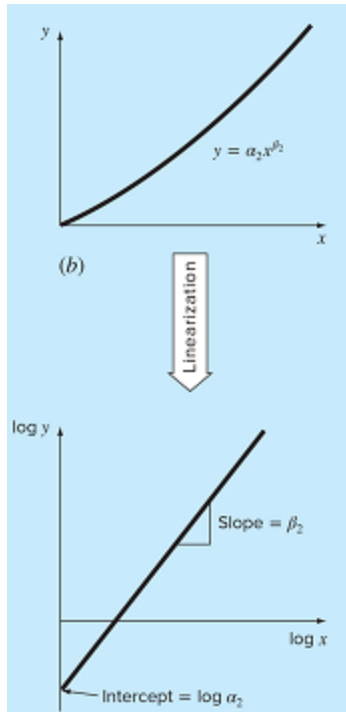


4.2 Curve Fitting [Least-Squares Regression for non-linear equations]

Example 4.2 – non-linear equation

Let's look at a power equation, and linearize as shown in the previous slide.

$$y = \alpha_2 x^{\beta_2}$$



$$Y = \beta_2 X + \log a_2$$

$$Y = \log y$$

$$X = \log x$$

			$y = \alpha_2 x^{\beta_2}$					
							Goodness of Fit	
			Transformed values				Sr	St
	xi	yi	Xi=log(xi)	Yi=log(yi)	Xi*Yi	Xi^2	(yi-ao-a1xi)^2	(yi-ybar)^2
	1	0.5	0.0	-0.3	0.0	0.0	0.0000	0.5318
	2	1.7	0.3	0.2	0.1	0.1	0.0000	0.0391
	3	3.4	0.5	0.5	0.3	0.2	0.0000	0.0107
	4	5.7	0.6	0.8	0.5	0.4	0.0000	0.1074
	5	8.4	0.7	0.9	0.6	0.5	0.0000	0.2461
sums ->		sums ->	2.1	2.1	1.4	1.2	0.0000	0.9350
		n	xbar	ybar				
		5	0.415836	0.42821				

4.2 Curve Fitting [Least-Squares Regression for non-linear equations]

Example 4.2 – non-linear equation

Let's look at a power equation, and linearize as shown in the previous slide.

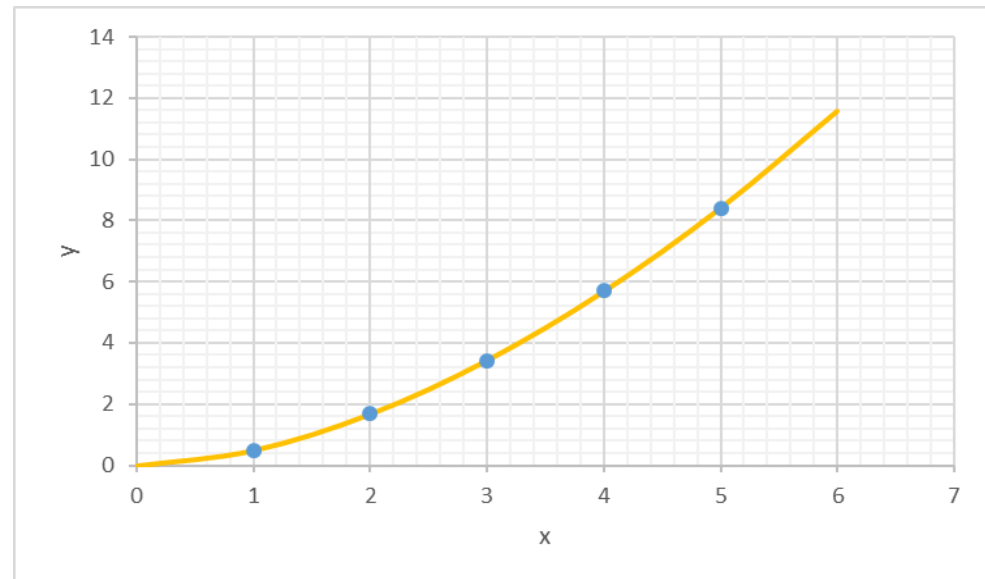
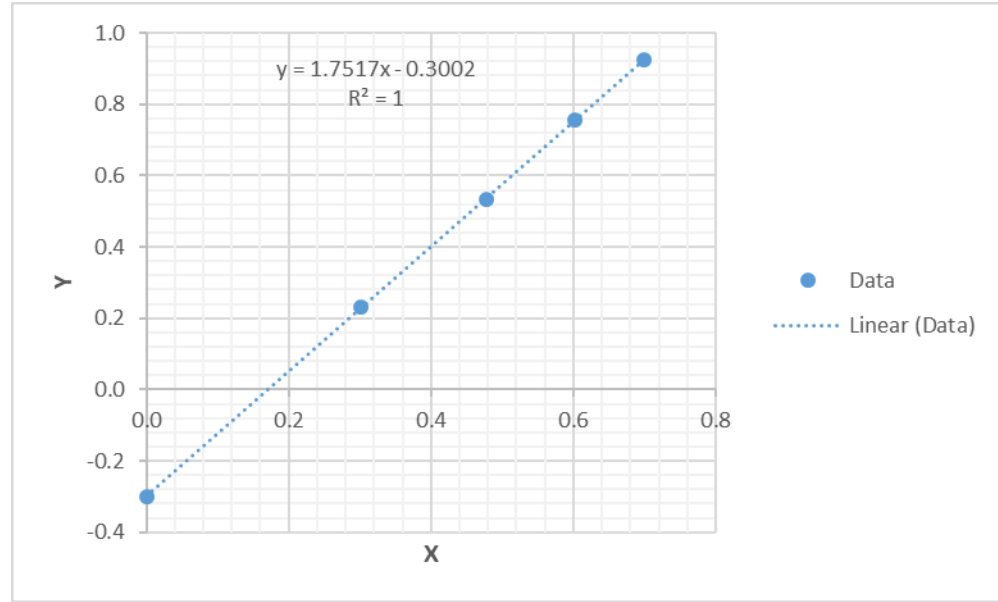
$$y = \alpha_2 x^{\beta_2}$$

$$y = \alpha_2 x^{\beta_2}$$

$$\alpha_2 = 0.501$$

$$\beta_2 = 1.752$$

$$r^2 = 0.999967$$



4.2 Curve Fitting [Least-Squares Regression for non-linear equations]

Other examples of linearization (from P. Teertstra)

EXAMPLE $y = a e^{bx}$, exponential function
take $\ln$ of both sides of equation
 $\ln y = \ln a + bx$
 $Y = A + bx$ ← linear, solve for A, b
 $y = (e^A) e^{bx}$ ← transform back

EXAMPLE $y = a x^b$ ← power-law, use base 10 $\log$
 $\log y = \log a + b \log x$
 $Y = A + bX$ ← linear, find A and b
 $y = (10^A) x^b$ ← transform back

EXAMPLE $y = a \left(\frac{x}{b+x} \right)$
 $\frac{1}{y} = \frac{1}{a} \left(\frac{b}{x} + 1 \right) = \frac{1}{a} + \frac{b}{a} \cdot \frac{1}{x}$
 $Y = A + B X$ ← linear, solve for A, B
 $y = \left(\frac{1}{A} \right) \left(\frac{X}{B/A + X} \right)$

4.3 Curve Fitting [Polynomial Regression]

Previously, we developed a method to fit a straight line to measured data. We extended this method to a few equation forms that could be linearized through a transformation (e.g. exponential and power functions).

Extending the least-squares criterion to fit polynomials is beneficial, since higher order polynomials provide a convenient method for fitting a variety of data.

Consider a second order polynomial:

$$y = a_0 + a_1x + a_2x^2 + e$$

And the sum of squares of residuals for 'n' data points:

$$S_r = \sum_{i=1}^n (y_i - a_0 - a_1x_i - a_2x_i^2)^2$$

Taking the derivative wrt the coefficients:

$$\frac{\partial S_r}{\partial a_0} = -2 \sum (y_i - a_0 - a_1x_i - a_2x_i^2)$$

$$\frac{\partial S_r}{\partial a_1} = -2 \sum x_i (y_i - a_0 - a_1x_i - a_2x_i^2)$$

$$\frac{\partial S_r}{\partial a_2} = -2 \sum x_i^2 (y_i - a_0 - a_1x_i - a_2x_i^2)$$

And setting equal to zero:

$$\begin{aligned} (n)a_0 + \left(\sum x_i\right)a_1 + \left(\sum x_i^2\right)a_2 &= \sum y_i \\ \left(\sum x_i\right)a_0 + \left(\sum x_i^2\right)a_1 + \left(\sum x_i^3\right)a_2 &= \sum x_i y_i \\ \left(\sum x_i^2\right)a_0 + \left(\sum x_i^3\right)a_1 + \left(\sum x_i^4\right)a_2 &= \sum x_i^2 y_i \end{aligned}$$

4.3 Curve Fitting [Polynomial Regression]

Therefore, to fit the second order (quadratic) polynomial to a data set of n points, we need to solve three simultaneous equations:

$$y = a_0 + a_1x + a_2x^2 + e$$

$$\begin{aligned}(n)a_0 + \left(\sum x_i\right)a_1 + \left(\sum x_i^2\right)a_2 &= \sum y_i \\ \left(\sum x_i\right)a_0 + \left(\sum x_i^2\right)a_1 + \left(\sum x_i^3\right)a_2 &= \sum x_iy_i \\ \left(\sum x_i^2\right)a_0 + \left(\sum x_i^3\right)a_1 + \left(\sum x_i^4\right)a_2 &= \sum x_i^2y_i\end{aligned}$$

First we determine the $[A]$ matrix and $\{b\}$ vector through summations as noted above.

We can use a method like Gauss-Seidel to solve for the coefficients (a_0, a_1, a_2) .

4.3 Curve Fitting [Polynomial Regression]

Example 4.3: Fitting a second order equation $y = a_0 + a_1x + a_2x^2 + e$

Step 1: Calculate the [A] matrix and {b} vector.

$$\begin{aligned}(n)a_0 + \left(\sum x_i\right)a_1 + \left(\sum x_i^2\right)a_2 &= \sum y_i \\ \left(\sum x_i\right)a_0 + \left(\sum x_i^2\right)a_1 + \left(\sum x_i^3\right)a_2 &= \sum x_i y_i \\ \left(\sum x_i^2\right)a_0 + \left(\sum x_i^3\right)a_1 + \left(\sum x_i^4\right)a_2 &= \sum x_i^2 y_i\end{aligned}$$

Step 2: Solve the linear equations for the coefficients.

[e.g. using Gaussian Elimination or Gauss-Seidel]

Step 3: Calculate the coefficient of determination (r^2).

Step 4: Plot the fitted function with the experimental data to verify the curve fit.

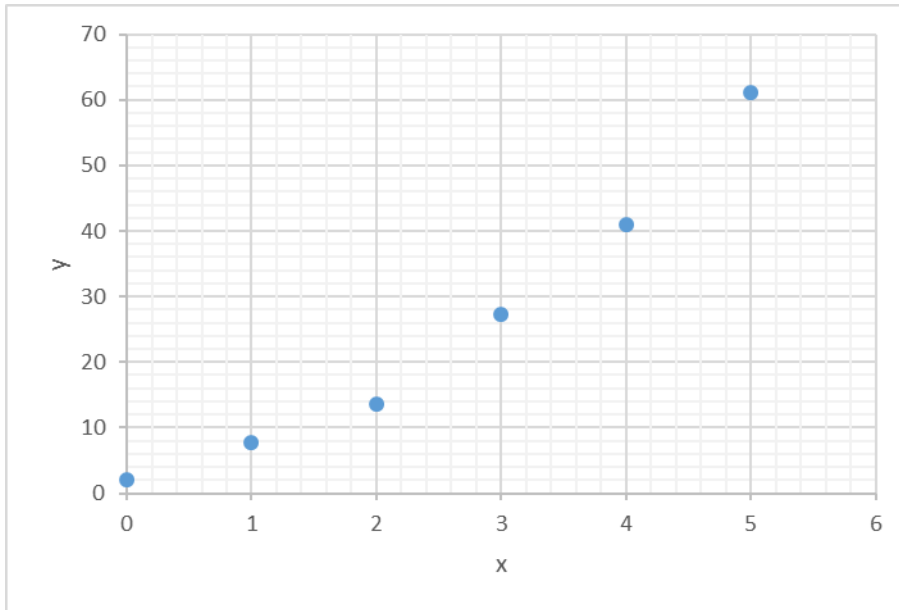
This process can be applied to polynomials of order 'm'

- The order 'm' of the polynomial must be less than the number of data points 'n'
- $n \geq m+1$

4.3 Curve Fitting [Polynomial Regression]

Example 4.3: Fitting a second order equation

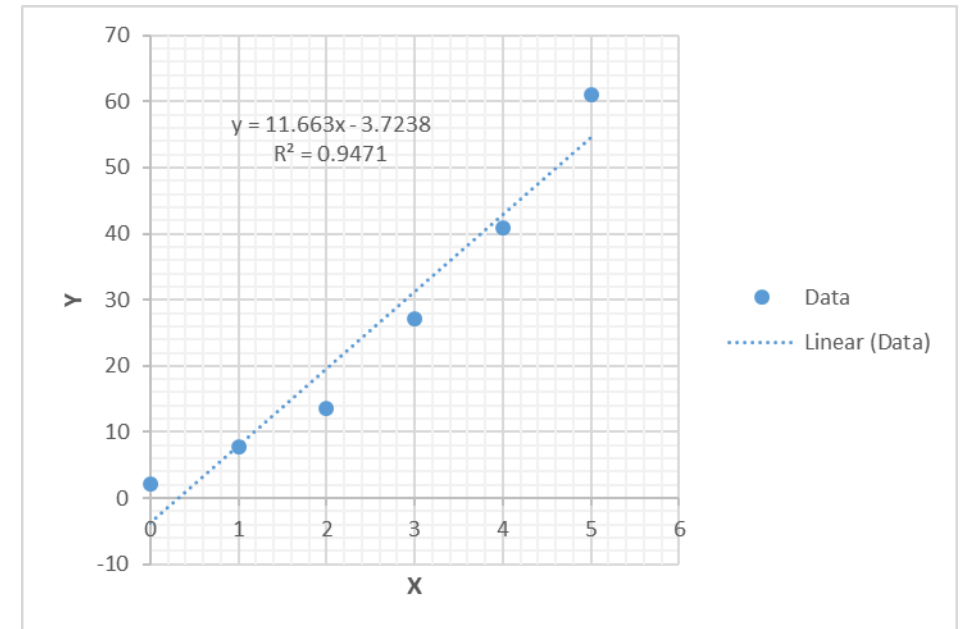
x_i	y_i
0	2.1
1	7.7
2	13.6
3	27.2
4	40.9
5	61.1



Aside: we can see that a linear regression gives a reasonable r^2 value, but does not fit the shape of the data very well.

Let's try:

$$y = a_0 + a_1x + a_2x^2 + e$$



4.3 Curve Fitting [Polynomial Regression]

Example 4.3: Fitting a second order equation

$$y = a_0 + a_1x + a_2x^2 + e$$

Step 1: Calculate the $[A]$ matrix and $\{b\}$ vector.

	<u>xi</u>	<u>yi</u>	<u>xi * yi</u>	<u>xi^2</u>	<u>xi^3</u>	<u>xi^4</u>	<u>xi^2yi</u>
	0	2.1	0.0	0.0	0.0	0.0	0.0
	1	7.7	7.7	1.0	1.0	1.0	7.7
	2	13.6	27.2	4.0	8.0	16.0	54.4
	3	27.2	81.6	9.0	27.0	81.0	244.8
	4	40.9	163.6	16.0	64.0	256.0	654.4
	5	61.1	305.5	25.0	125.0	625.0	1527.5
sums ->	15.0	152.6	585.6	55.0	225.0	979.0	2488.8
n	xbar	ybar					
6	2.5	25.43333					
		A matrix			Initial Guesses		
		$(n)a_0 + (\sum x_i)a_1 + (\sum x_i^2)a_2 = \sum y_i$			a0	b1	
		$(\sum x_i)a_0 + (\sum x_i^2)a_1 + (\sum x_i^3)a_2 = \sum x_i y_i$			a1	b2	
		$(\sum x_i^2)a_0 + (\sum x_i^3)a_1 + (\sum x_i^4)a_2 = \sum x_i^2 y_i$			a2	b3	
		6	15.0	55.0	1	152.6	
		15.0	55.0	225.0	1	585.6	
		55.0	225.0	979.0	1	2488.8	

4.3 Curve Fitting [Polynomial Regression]

Example 4.3: Fitting a second order equation

Step 2: Solve the linear equations for the coefficients.
[Gauss-Seidel]

Gauss Seidel w/ Relaxation							
	a (col1)	a (col2)	a (col3)	{x} guesses	{b}		
Row1	6	15	55	1	152.6		
Row2	15	55	225	1	585.6		
Row3	55	225	979	1	2488.8		
Relaxation (w) ->	1.00						
Convergence ->	1.00E-06						
					Convergence ->		1.00E-06
Iteration #	x1	x2	x3		εa1	εa2	εa3
0	1.000000000000	1.000000000000	1.000000000000				
1	13.766666666667	2.80181818182	1.12484600860	-	9.274E+01	6.431E+01	1.110E+01
2	8.11769946658	3.83171192846	1.20550699227	-	6.959E+01	2.688E+01	6.691E+00
3	4.80357274969	4.40558791806	1.25980206047	-	6.899E+01	1.303E+01	4.310E+00
4	2.87117798384	4.71048848429	1.29828937888	-	6.730E+01	6.473E+00	2.964E+00
5	1.75612614954	4.85714540925	1.32722711409	-	6.349E+01	3.019E+00	2.180E+00
6	1.12422126438	4.91110146116	1.35032686588	-	5.621E+01	1.099E+00	1.711E+00
7	0.77758340985	4.91114007326	1.36979202858	-	4.458E+01	7.862E-04	1.421E+00
506	2.47857077182	2.35928647459	1.86071414787	Converged	8.125E-07	9.882E-07	2.272E-07

$$y = a_0 + a_1x + a_2x^2.$$

4.3 Curve Fitting [Polynomial Regression]

Example 4.3: Fitting a second order equation

Step 3: Calculate the coefficient of determination (r^2).

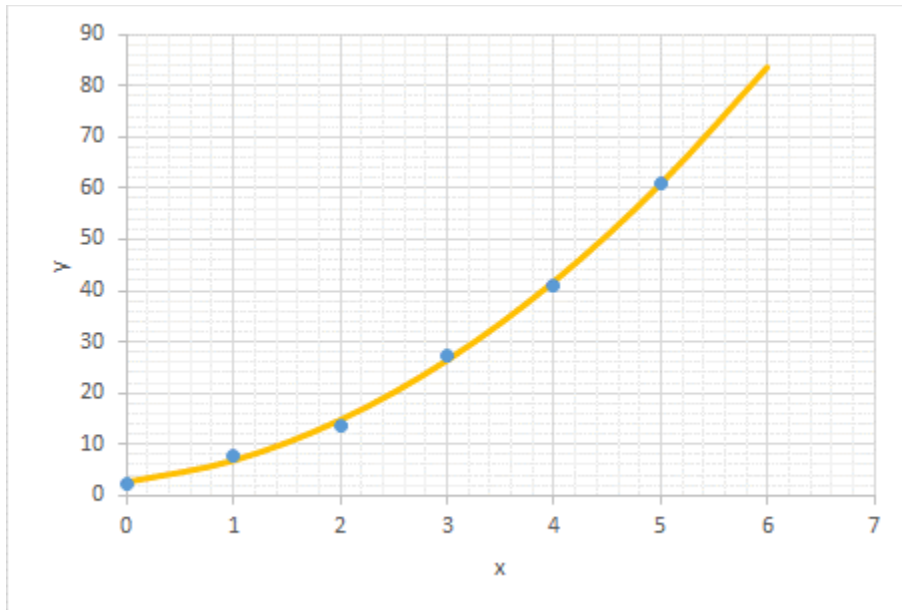
								Goodness of Fit			Values from fit equation	
	<u>xi</u>	<u>yi</u>	<u>xi * yi</u>	<u>xi^2</u>	<u>xi^3</u>	<u>xi^4</u>	<u>xi^2*yi</u>	<u>Sr</u>	<u>St</u>		<u>x</u>	<u>y</u>
	0	2.1	0.0	0.0	0.0	0.0	0.0	0.1433	544.4444		0	2.479
	1	7.7	7.7	1.0	1.0	1.0	7.7	1.0029	314.4711		1	6.699
	2	13.6	27.2	4.0	8.0	16.0	54.4	1.0816	140.0278		2	14.640
	3	27.2	81.6	9.0	27.0	81.0	244.8	0.8049	3.1211		3	26.303
	4	40.9	163.6	16.0	64.0	256.0	654.4	0.6196	239.2178		4	41.687
	5	61.1	305.5	25.0	125.0	625.0	1527.5	0.0943	1272.1111		5	60.793
											6	83.620
sums ->	15.0	152.6	585.6	55.0	225.0	979.0	2488.8	3.7466	2513.3933			
n	xbar	ybar										
6	2.5	25.43333										
		A matrix					Initial Guesses					
		(n)a ₀ + (Σ x _i)a ₁ + (Σ x _i ²)a ₂ = Σ y _i	a ₀	b ₁								
		(Σ x _i)a ₀ + (Σ x _i ²)a ₁ + (Σ x _i ³)a ₂ = Σ x _i y _i	a ₁	b ₂								
		(Σ x _i ²)a ₀ + (Σ x _i ³)a ₁ + (Σ x _i ⁴)a ₂ = Σ x _i ² y _i	a ₂	b ₃								
	6	15.0	55.0	1	152.6							
	15.0	55.0	225.0	1	585.6							
	55.0	225.0	979.0	1	2488.8							
r^2	0.998509		Gauss-Seidel solution									
r	0.999254		a ₀	2.47857077								
			a ₁	2.35928647								
			a ₂	1.86071415								

$$r^2 = \frac{S_t - S_r}{S_t}$$

4.3 Curve Fitting [Polynomial Regression]

Example 4.3: Fitting a second order equation

Step 4: Plot the fitted function with the experimental data to verify the curve fit.



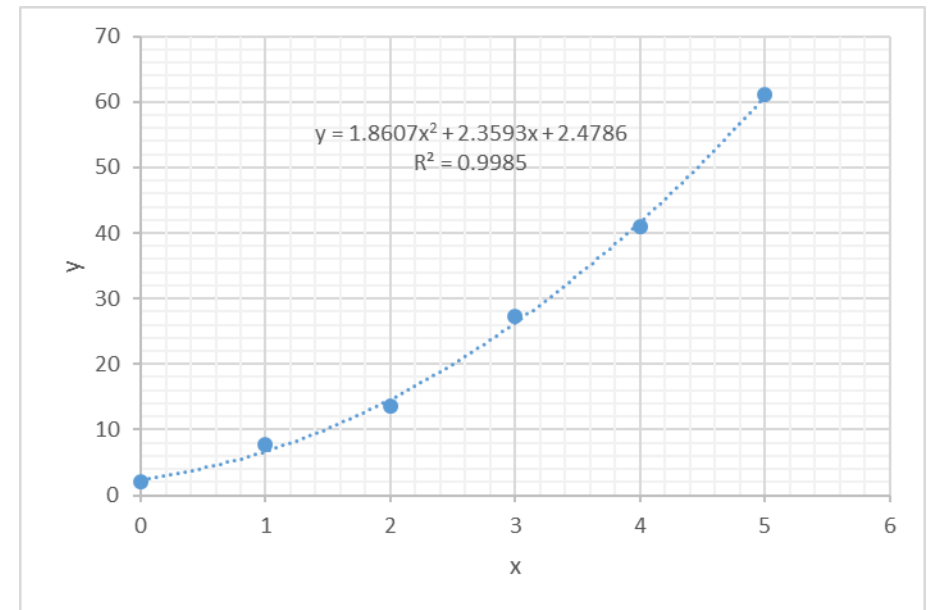
$$y = a_0 + a_1x + a_2x^2$$

Gauss-Seidel solution

a0	2.47857077
a1	2.35928647
a2	1.86071415

r^2	0.998509
r	0.999254

Using polynomial trend line in Excel



- 99.85% of the uncertainty is explained by the curve fit (this is a good curve fit!)

4.4 Polynomial Interpolation

Section 18.1 in Chapra 8th edition

Introduction

In many cases, we have a set of data, and are looking to determine values between the measured/reported data points. This is known as interpolation.

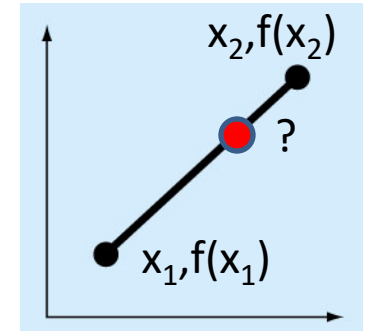
In general, we can fit a polynomial of order 'n' to 'n+1' data points. For example, there is one straight line that connects two points.

$$f(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$$



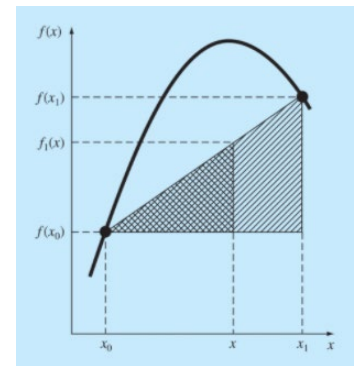
$$f(x) = a_0 + a_1x$$

Linear interpolation for two data points



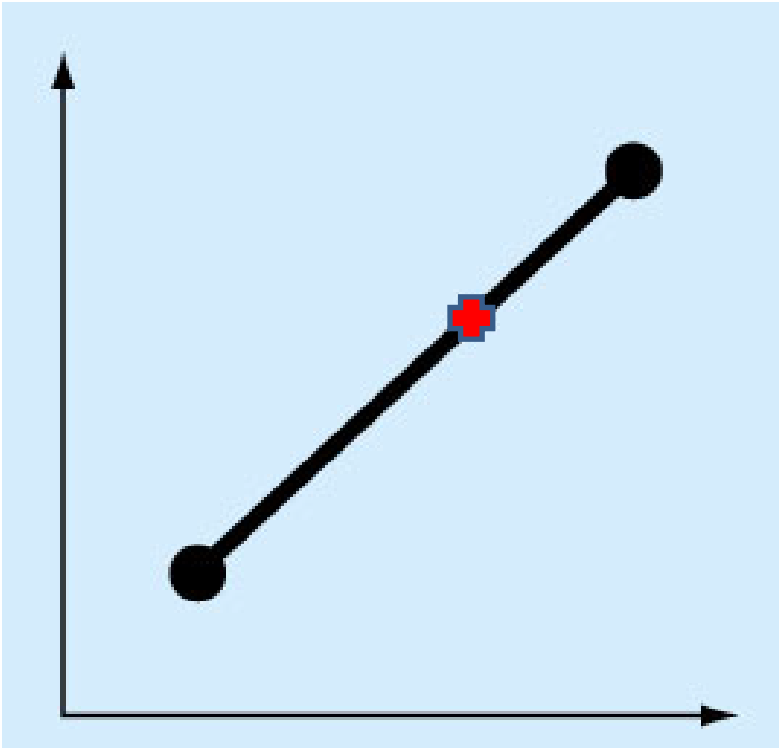
Two general interpolation methods:

1. Newton's divided difference interpolating polynomials (we will focus on this one)
2. Lagrange interpolating polynomials (not covered, section 18.2 if interested)



4.4 Polynomial Interpolation [Newton's divided difference]

The most simple interpolation is linear $f(x) = a_0 + a_1x$

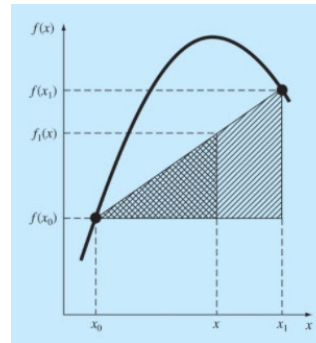


$$\frac{f_1(x) - f(x_0)}{x - x_0} = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$f_1(x) = f(x_0) + \frac{f(x_1) - f(x_0)}{x_1 - x_0} (x - x_0)$$



This approach will estimate an intermediate value between 2 data points.



* We are evaluating x in the first interval, i.e. $x_0 \leq x \leq x_1$. This is known as the forward divided difference. We can add more terms by adding more data points beyond the first interval.

4.4 Polynomial Interpolation

Let's consider a 2nd order fit to 3 data points

$$f(x) = a_0 + a_1x + a_2x^2$$

$$a_0 = b_0 - b_1x_0 + b_2x_0x_1$$

$$a_1 = b_1 - b_2x_0 - b_2x_1$$

$$a_2 = b_2$$

or rewriting:

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$

$f_2(x)$ -> predicted value

$$b_0 = f(x_0)$$

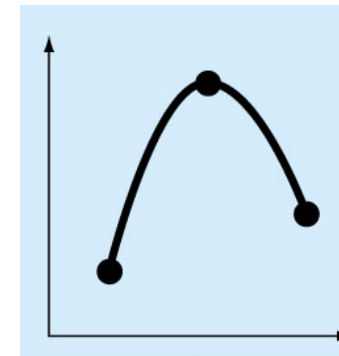
$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

$$b_2 = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$

* We generally get improved estimates when the bounds are moved closer to the target point

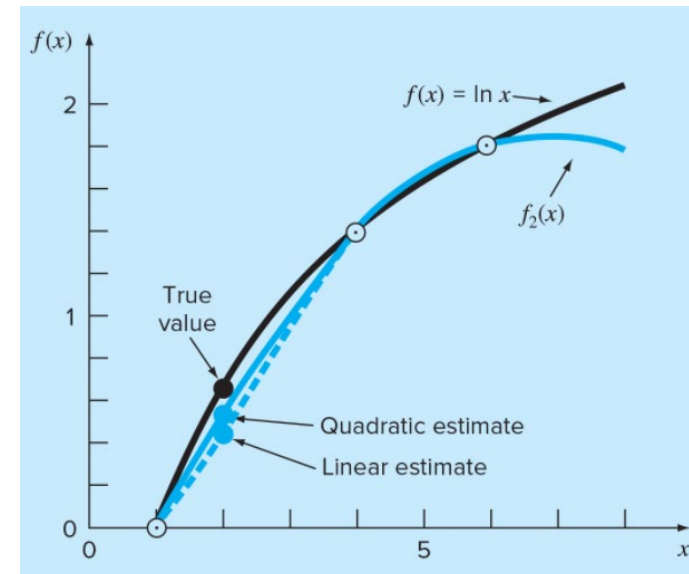
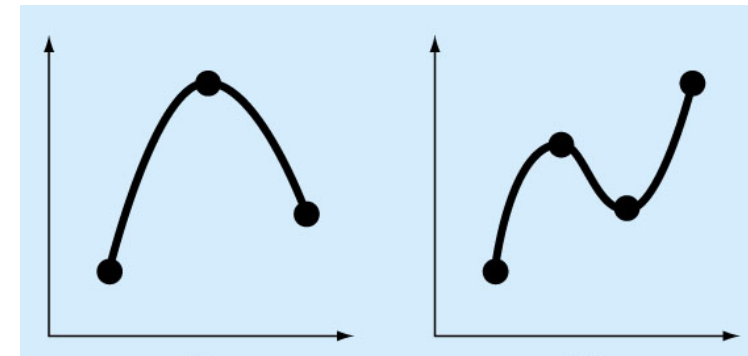
2nd order

3 data points



3rd order

4 data points



$$\ln(2) = 0.6931472$$

Linear
 $f_1(2) = 0.3583519$

2nd order
 $f_2(2) = 0.5658444$

4.4 Polynomial Interpolation

Aside, consider the representation:

$$f(x) = a_0 + a_1x + a_2x^2$$

Can be written as:

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$

$f_2(x)$ -> predicted value

Multiplying the terms of $f_2(x)$:

$$f_2(x) = b_0 + b_1x - b_1x_0 + b_2x^2 + b_2x_0x_1 - b_2xx_0 - b_2xx_1$$

Or writing in the original form:

$$f(x) = a_0 + a_1x + a_2x^2$$

Where:

$$a_0 = b_0 - b_1x_0 + b_2x_0x_1$$

$$a_1 = b_1 - b_2x_0 - b_2x_1$$

$$a_2 = b_2$$

Applying boundary conditions to:

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$

i) For $x=x_0$:

$$b_0 = f(x_0)$$

ii) For $x=x_1$ and substituting b_0 :

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

iii) For $x=x_2$ and substituting b_0 and b_1 :

$$b_2 = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$

4.4 Polynomial Interpolation

Example 4.4: Fit a 2nd order polynomial interpolation to the following points:

$$x_0 = 1 \quad f(x_0) = 0$$

$$x_1 = 4 \quad f(x_1) = 1.386294$$

$$x_2 = 6 \quad f(x_2) = 1.791759$$

$$b_0 = f(x_0)$$

$$b_0 = 0$$

$$b_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

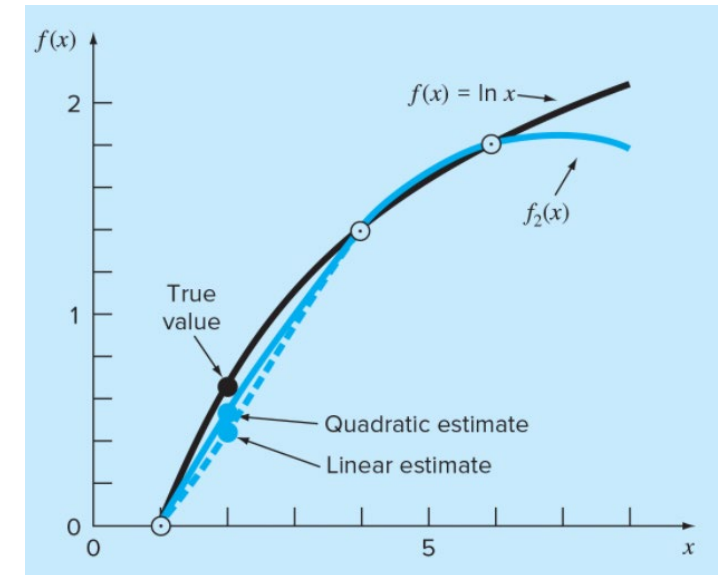
$$b_1 = \frac{1.386294 - 0}{4 - 1} = 0.4620981$$

$$b_2 = \frac{\frac{f(x_2) - f(x_1)}{x_2 - x_1} - \frac{f(x_1) - f(x_0)}{x_1 - x_0}}{x_2 - x_0}$$

$$b_2 = \frac{\frac{1.791759 - 1.386294}{6 - 4} - 0.4620981}{6 - 1} = -0.0518731$$

$$f_2(x) = 0 + 0.4620981(x - 1) - 0.0518731(x - 1)(x - 4)$$

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$



4.4 Polynomial Interpolation

Example 4.4:

$$f_2(x) = 0 + 0.4620981(x - 1) - 0.0518731(x - 1)(x - 4)$$

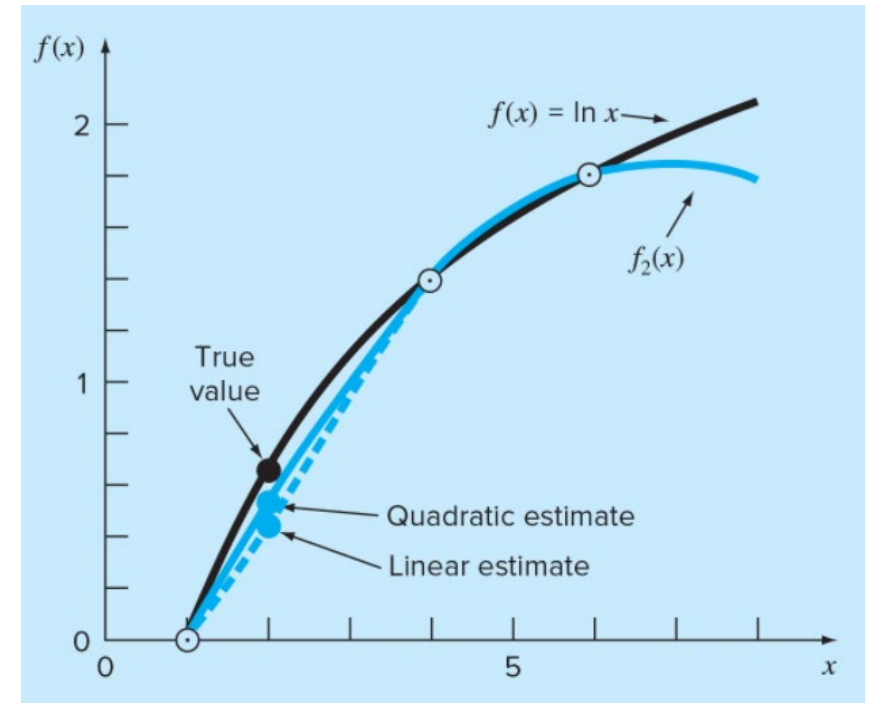
Evaluating at $x=2$

$$f_2(2) = 0.5658444$$

and comparing to $\ln(x)$, $\ln(2) = 0.6932$

$$\varepsilon_t = 18.4\%$$

$$f_2(x) = b_0 + b_1(x - x_0) + b_2(x - x_0)(x - x_1)$$



How can we improve accuracy?

1. Add more terms to the polynomial (section 18.1.3). e.g. 3rd order gives:
2. Use a different interpolation scheme (splines, next section)

$$\varepsilon_t = 9.3\%$$

4.5 Spline Interpolation [General]

Polynomial interpolation – an n^{th} order polynomial directly fits $n+1$ data points (curve passes through all points).

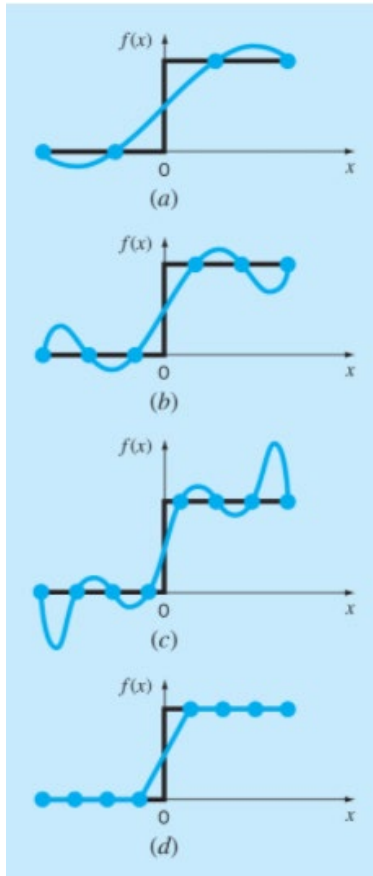
If the underlying curve (actual response) is an n^{th} order polynomial, a polynomial based on $n+1$ data points will provide exact results (at the data points, and between the data points).

If we are approximating a different underlying response (e.g. $\ln(x)$) with a polynomial, the polynomial will pass through the fitted data points, but will only approximate the response between the data points.

4.5 Spline Interpolation [General]

Challenges with polynomial interpolation include error in our prediction due to round off, and overshoot.

Consider a polynomial fit to a step function:



3rd order polynomial (requires 4 data points)

5th order polynomial (requires 6 data points)

7th order polynomial – notice the oscillations, and significant overshoot (requires 8 data points)

We are not
always
improving
with higher
order
polynomials

Splines provide an alternate approach to interpolation that addresses some of the limitations of polynomial interpolation.



4.5 Spline Interpolation [General]

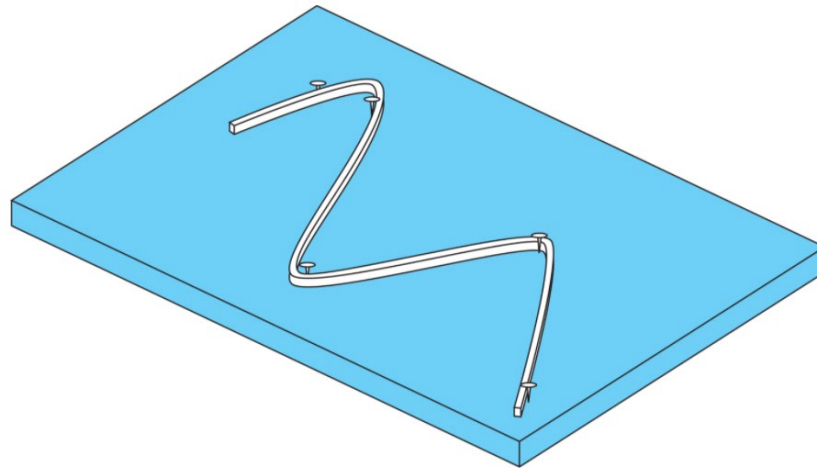
A spline is a lower order polynomial applied to a subset of points in the data set.

Spline functions are characterized by the order of the fit.

For example, a 3rd order polynomial is called a 'cubic spline'. This is the most widely used version in engineering (provides continuity of 1st and 2nd derivative).

The concept originated from using a thin, flexible strip of material to join points (e.g. pins in a board).

This is called a natural spline. Note that the slope of the flexible strip is constant at the end points.



4.5 Spline Interpolation [Linear]

Linear, first order spline fit

For n data points, there are $i = 1$ to $n-1$ intervals (spaces between data points).

Each interval can be described by a function.

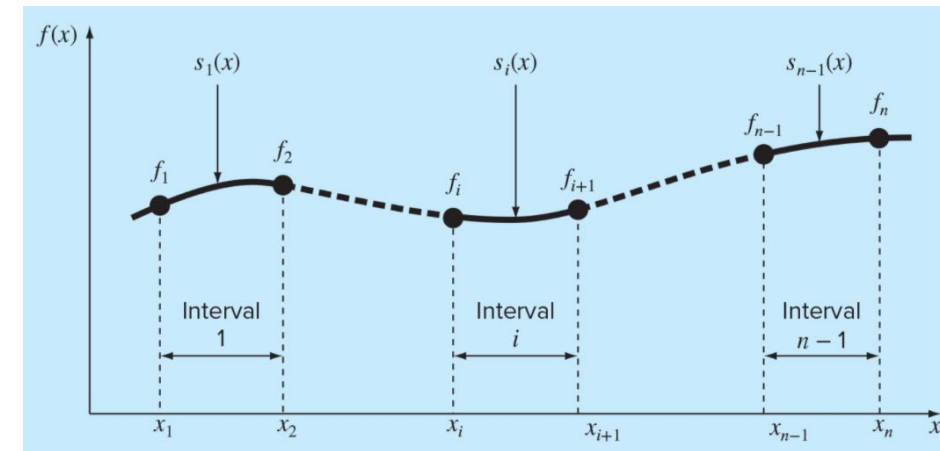
For linear interpolation, each interval can be described as:

$$s_i(x) = a_i + b_i(x - x_i) \quad \text{Valid for } x_i \leq x \leq x_{i+1}$$

$$a_i = f_i \quad \text{[intercept]}$$

$$b_i = \frac{f_{i+1} - f_i}{x_{i+1} - x_i} \quad \text{[slope]}$$

$$\Rightarrow s_i(x) = f_i + \frac{f_{i+1} - f_i}{x_{i+1} - x_i} (x - x_i)$$



- Note that we are joining points using straight lines (linear spline) (Same as linear interpolation)

4.5 Spline Interpolation [Linear]

Example 4.5: Fit a linear spline to the following data to estimate the function value at $x=5$

i	x_i	$f(x_i)$
1.0	3.0	2.5
2.0	4.5	1.0
3.0	7.0	2.5
4.0	9.0	0.5

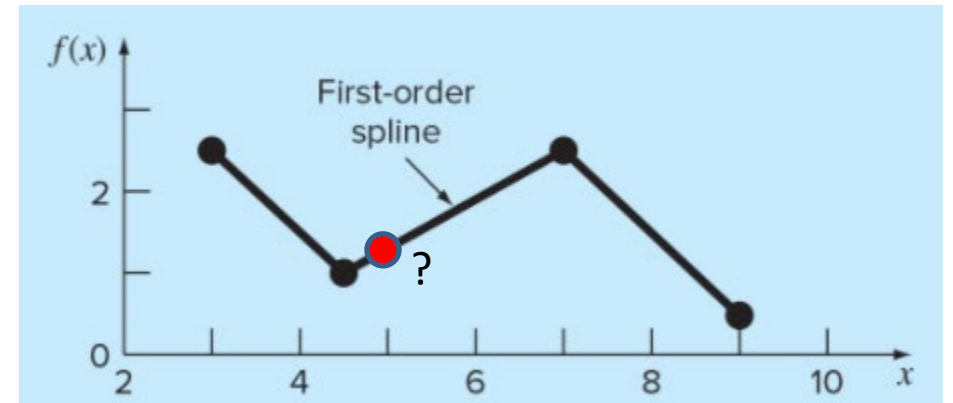
$$s_i(x) = a_i + b_i(x - x_i)$$

for $4.5 \leq x \leq 7$

$$s_2(x) = 1.0 + \frac{2.5 - 1.0}{7.0 - 4.5}(x - 4.5)$$

for $x = 5$

$$s_2(x) = 1.0 + \frac{2.5 - 1.0}{7.0 - 4.5}(5 - 4.5) = 1.3$$



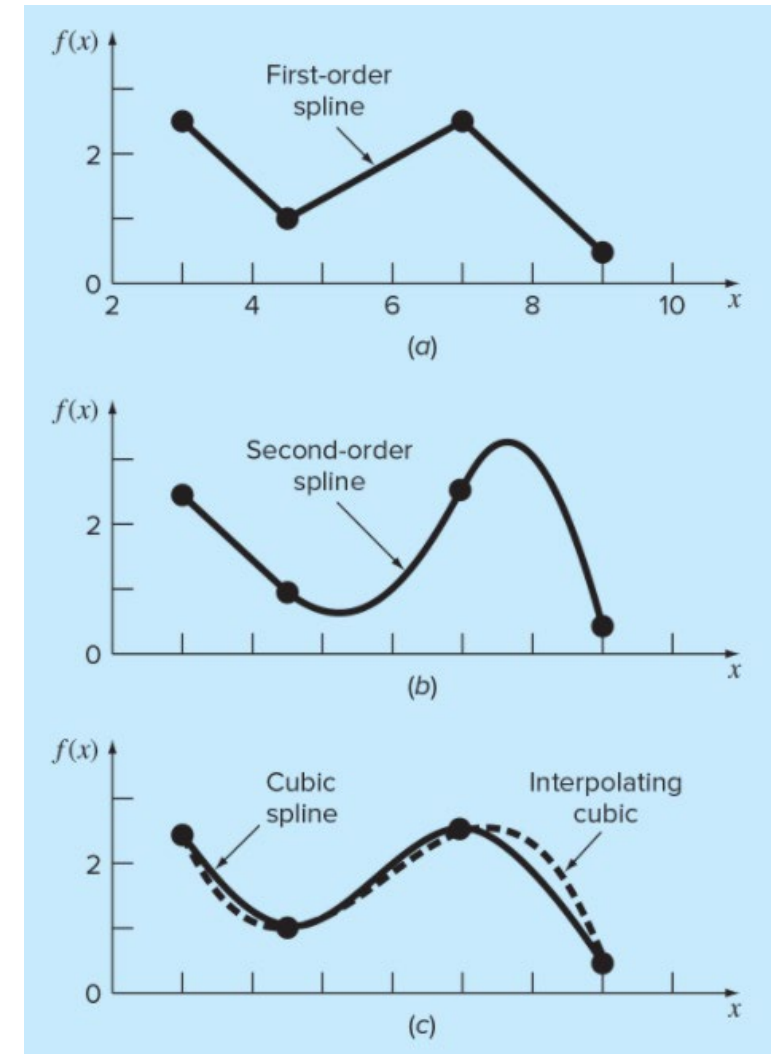
* Note that we are joining points using straight lines (linear spline)

4.5 Spline Interpolation [Higher order]

A disadvantage of linear (first-order) splines is that the transitions at data points, also known as knots, are abrupt. There is no continuity of slope at the data points.

Smooth transitions at knots requires that the function value, and the derivative(s) be equal at each data point.

For example, a second-order spline is continuous in magnitude and first derivative at the data points.



(a) Linear spline, (b) quadratic spline, and (c) cubic spline

Also shown is a cubic interpolating polynomial (previous section)

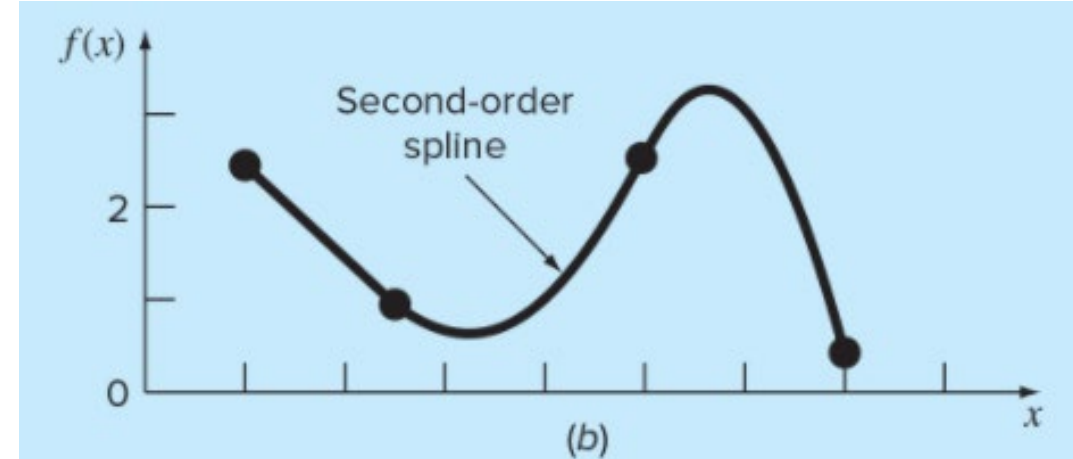
4.5 Spline Interpolation [Quadratic or second-order]

Quadratic or second-order splines

A second-order polynomial describing an interval:

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2$$

For n data points, there are $n-1$ intervals and therefore $3*(n-1)$ constants (a , b , c)



Aside:

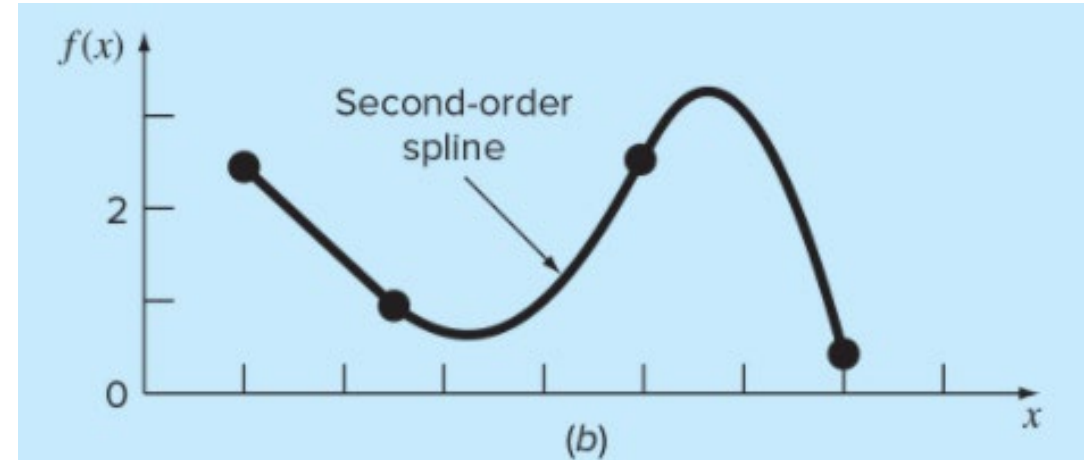
- Note that we are fitting a spline to each interval separately. We have a new set of constants a_i , b_i , c_i for each interval.
- A polynomial interpolation fit one curve to all data points (i.e. over 2 intervals for a quadratic, 3 intervals for a cubic)

4.5 Spline Interpolation [Quadratic or second-order]

Quadratic or second-order splines

$$s_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2$$

For n data points, there are $n-1$ intervals and therefore $3*(n-1)$ constants (a_i, b_i, c_i)



At a given interval,

$$a_i = f_i \quad \text{so} \quad s_i(x) = f_i + b_i(x - x_i) + c_i(x - x_i)^2$$

So we need to determine $2*(n-1)$ constants (b_i, c_i)

The functions must be equal at both knots (e.g. $i+1$)

$$f_i + b_i(x_{i+1} - x_i) + c_i(x_{i+1} - x_i)^2 = f_{i+1} + b_{i+1}(x_{i+1} - x_{i+1}) + c_{i+1}(x_{i+1} - x_{i+1})^2$$

4.5 Spline Interpolation [Quadratic or second-order]

For $f_i + b_i (x_{i+1} - x_i) + c_i (x_{i+1} - x_i)^2 = f_{i+1} + b_{i+1} (x_{i+1} - x_{i+1}) + c_{i+1} (x_{i+1} - x_{i+1})^2$

Let $h_i = x_{i+1} - x_i$

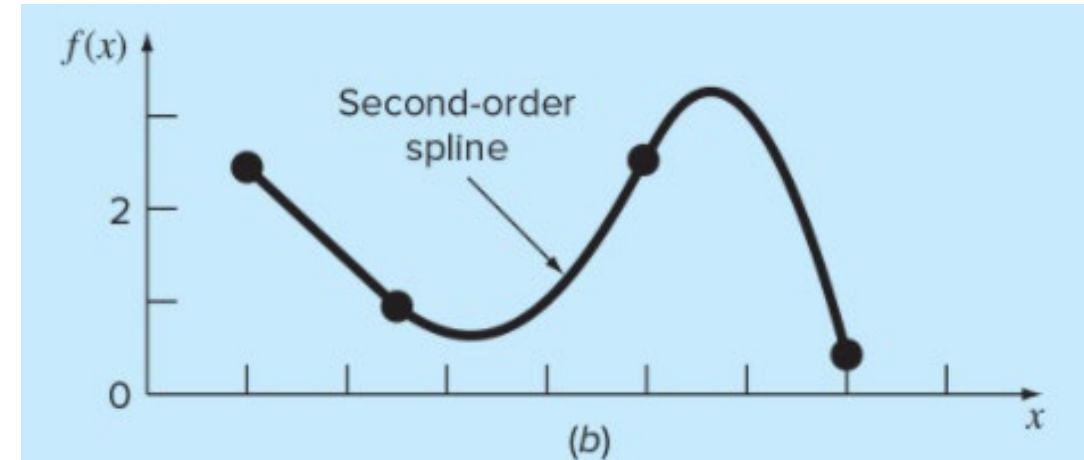
Then $f_i + b_i h_i + c_i h_i^2 = f_{i+1}$

For $i = 1$ to $n-1$, this gives us $n-1$ equations.

We still need an additional $n-1$ equations to solve for the coefficients.

Enforce continuity of first derivative (slope) at the interior points. For each interval:

$$s'_i(x) = b_i + 2c_i (x - x_i)$$



4.5 Spline Interpolation [Quadratic or second-order]

$$s'_i(x) = b_i + 2c_i(x - x_i)$$

At point $i+1$, continuity of slope can be written as:

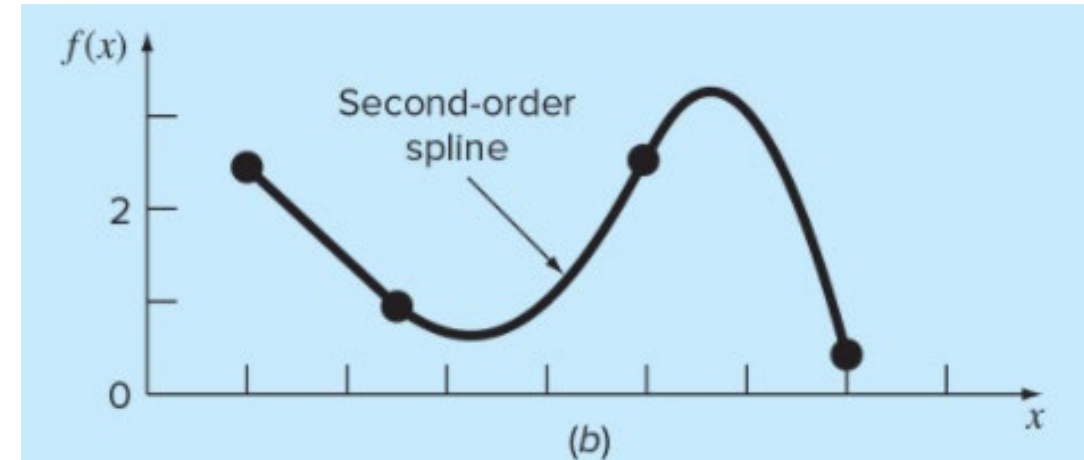
$$b_i + 2c_i h_i = b_{i+1} \quad h_i = x_{i+1} - x_i$$

This gives us $n-2$ additional equations (interior points).

We need one more equation. In the absence of additional information, we can assume that the second derivative is zero at the first data point. This is a common approach to generate the last equation, and results in:

$$c_1 = 0$$

This condition implies that the first 2 points are connected by a straight line.



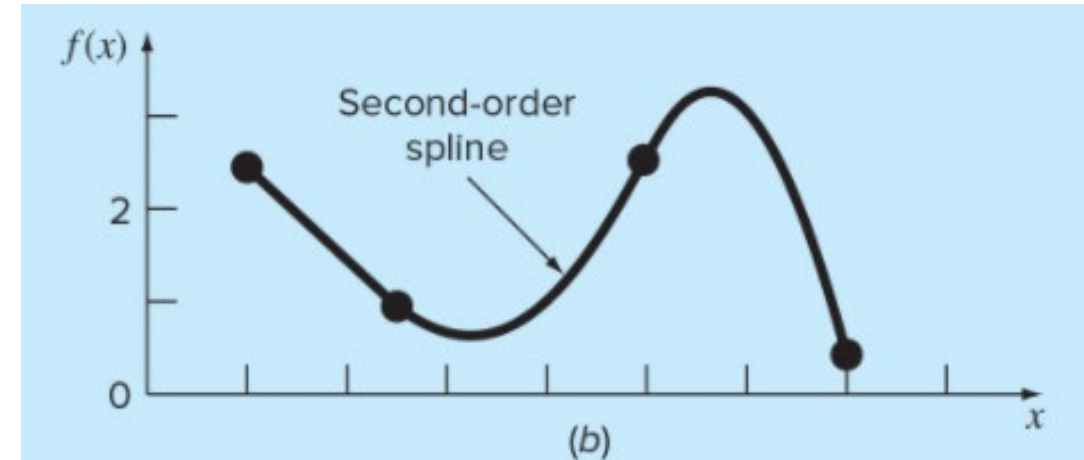
4.5 Spline Interpolation [Quadratic or second-order]

Example 4.5b: Fit a quadratic spline to the following data to estimate the function value at $x=5$.

i	x_i	$f(x_i)$
1.0	3.0	2.5
2.0	4.5	1.0
3.0	7.0	2.5
4.0	9.0	0.5

We have 4 data points and 3 intervals.

For reference, we need $2*(n-1)-1$ equations, if we enforce continuity at knots, and $c_1=0$ (zero second derivative).



4.5 Spline Interpolation [Quadratic or second-order]

i	x_i	$f(x_i)$
1.0	3.0	2.5
2.0	4.5	1.0
3.0	7.0	2.5
4.0	9.0	0.5

Example 4.5b

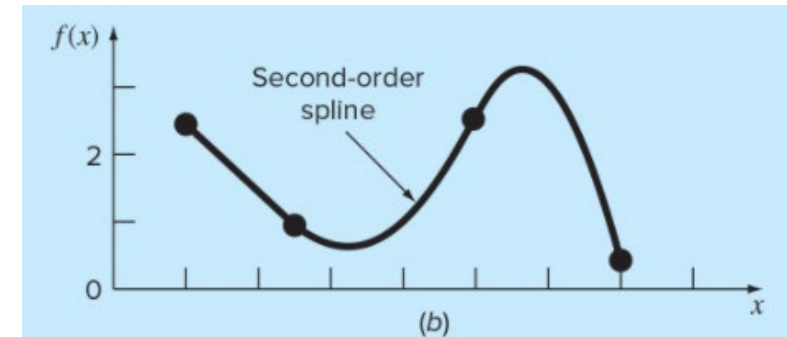
For $i = 1$ to 3:

$$f_1 + b_1 h_1 = f_2$$

$$f_2 + b_2 h_2 + c_2 h_2^2 = f_3$$

$$f_3 + b_3 h_3 + c_3 h_3^2 = f_4$$

Continuity at each data point



For $c_1 = 0$ (2nd derivative at first data point assumed to be zero) and continuity of first derivative at the 2 interior points:

$$b_1 = b_2$$

$$b_2 + 2c_2 h_2 = b_3$$

* This gives us the full set of equations to solve for the coefficients.

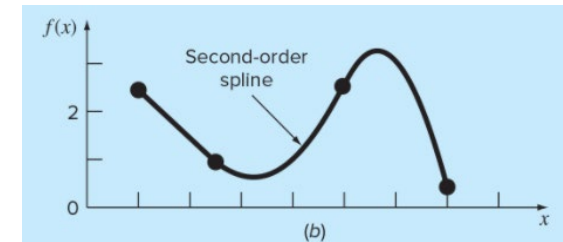
4.5 Spline Interpolation [Quadratic or second-order]

Example 4.5b

The function and interval values are:

$$\begin{array}{ll} f_1 = 2.5 & h_1 = 4.5 - 3.0 = 1.5 \\ f_2 = 1.0 & h_2 = 7.0 - 4.5 = 2.5 \\ f_3 = 2.5 & h_3 = 9.0 - 7.0 = 2.0 \\ f_4 = 0.5 \end{array}$$

i	x _i	f(x _i)
1.0	3.0	2.5
2.0	4.5	1.0
3.0	7.0	2.5
4.0	9.0	0.5



Substituting into our equations:

$$\begin{bmatrix} 1.5 & 0 & 0 & 0 & 0 \\ 0 & 2.5 & 6.25 & 0 & 0 \\ 0 & 0 & 0 & 2 & 4 \\ 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & 5 & -1 & 0 \end{bmatrix} \begin{Bmatrix} b_1 \\ b_2 \\ c_2 \\ b_3 \\ c_3 \end{Bmatrix} = \begin{Bmatrix} -1.5 \\ 1.5 \\ -2 \\ 0 \\ 0 \end{Bmatrix}$$

$$f_1 + b_1 h_1 = f_2 \longrightarrow b_1 h_1 = f_2 - f_1$$

$$f_2 + b_2 h_2 + c_2 h_2^2 = f_3$$

$$f_3 + b_3 h_3 + c_3 h_3^2 = f_4$$

$$b_1 = b_2 \longrightarrow b_1 - b_2 = 0$$

$$b_2 + 2c_2 h_2 = b_3$$

4.5 Spline Interpolation [Quadratic or second-order]

Example 4.5b

Solving

$$\begin{bmatrix} 1.5 & 0 & 0 & 0 & 0 \\ 0 & 2.5 & 6.25 & 0 & 0 \\ 0 & 0 & 0 & 2 & 4 \\ 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & 5 & -1 & 0 \end{bmatrix} \begin{Bmatrix} b_1 \\ b_2 \\ c_2 \\ b_3 \\ c_3 \end{Bmatrix} = \begin{Bmatrix} -1.5 \\ 1.5 \\ -2 \\ 0 \\ 0 \end{Bmatrix}$$

Gives:

$$\begin{aligned} b_1 &= -1 \\ b_2 &= -1 & c_2 &= 0.64 \\ b_3 &= 2.2 & c_3 &= -1.6 \end{aligned}$$

Then:

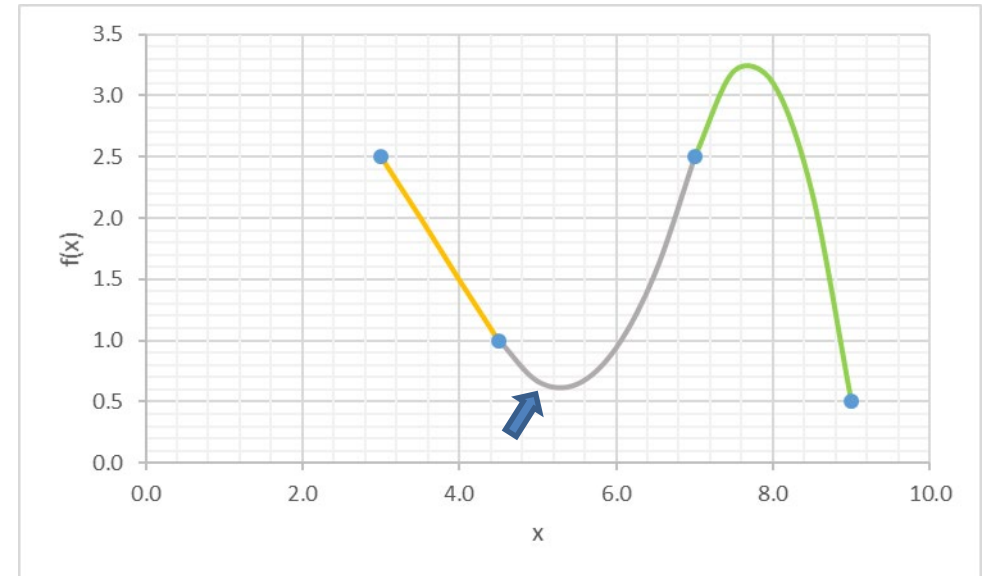
$$\begin{aligned} s_1(x) &= 2.5 - (x - 3) \\ s_2(x) &= 1.0 - (x - 4.5) + 0.64(x - 4.5)^2 \\ s_3(x) &= 2.5 + 2.2(x - 7.0) - 1.6(x - 7.0)^2 \end{aligned}$$

For $x=5$, this is in the second interval (s_2)

$$s_2(5) = 1.0 - (5 - 4.5) + 0.64(5 - 4.5)^2 = 0.66$$

x	s1	s2	s3
3.00	2.50		
3.50	2.00		
4.00	1.50		
4.50	1.00	1.00	
5.00		0.66	
5.50		0.64	
6.00		0.94	
6.50		1.56	
7.00		2.50	2.50
7.50			3.20
8.00			3.10
8.50			2.20
9.00			0.50

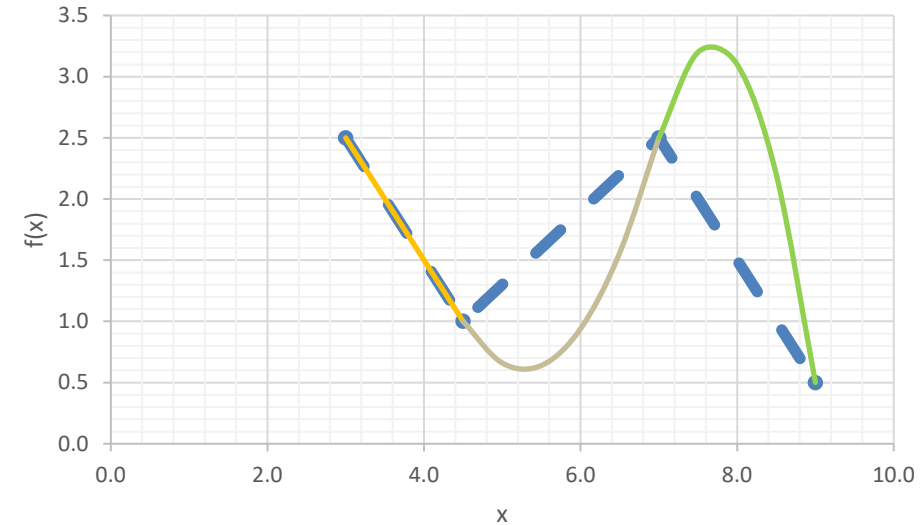
Recall, $c_1=0$



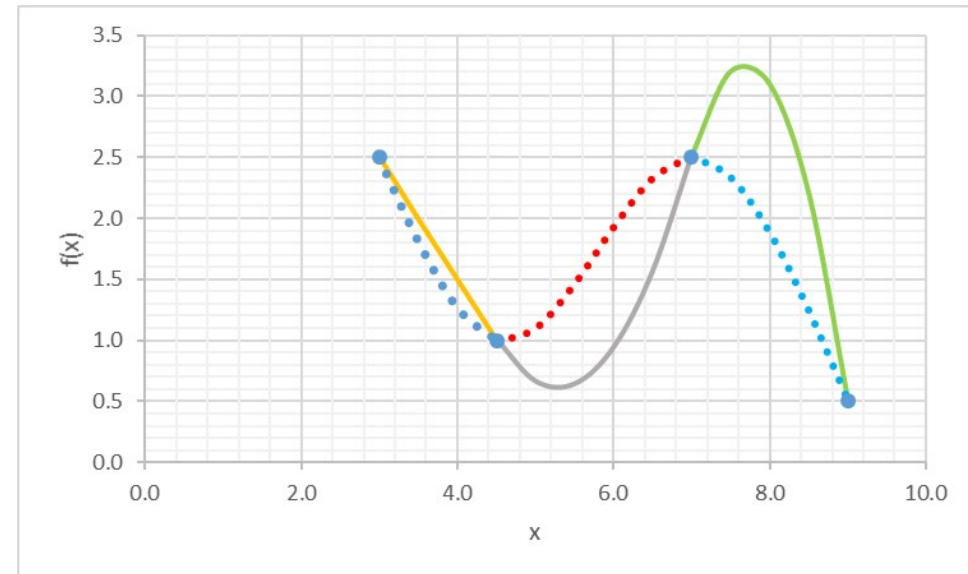
4.5 Spline Interpolation

- A spline is a piecewise (interval) fit to the data.
- Splines are often preferred over polynomials since a lower order function provides similar results and avoids issues with higher order polynomials (oscillation at the edges of an interval).
- We only used 4 points in our example, splines readily handle large numbers of data points.

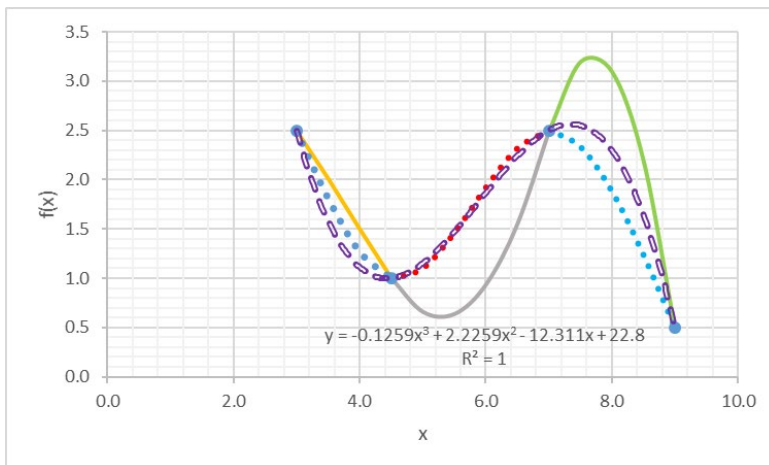
Linear compared to quadratic spline



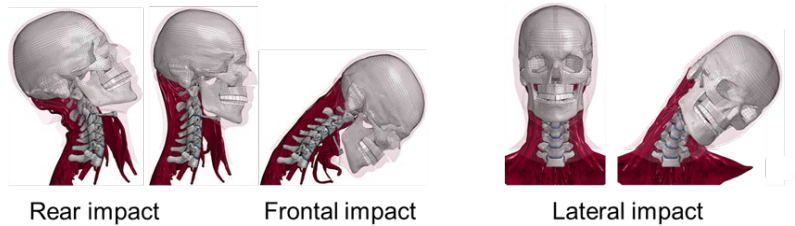
Cubic compared to quadratic spline



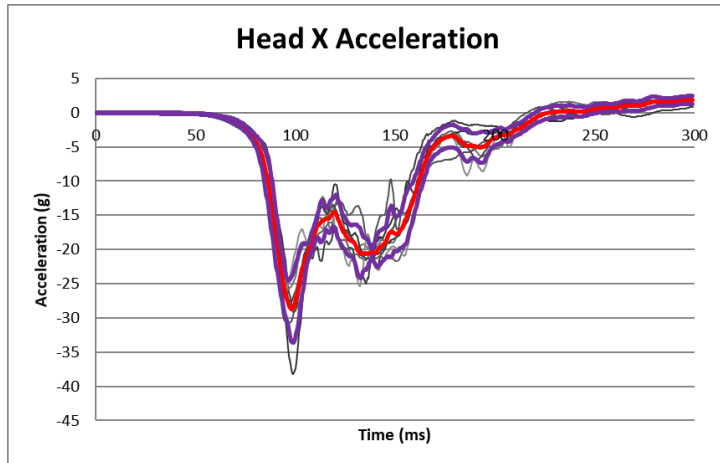
Cubic polynomial (purple) vs quadratic and cubic spline



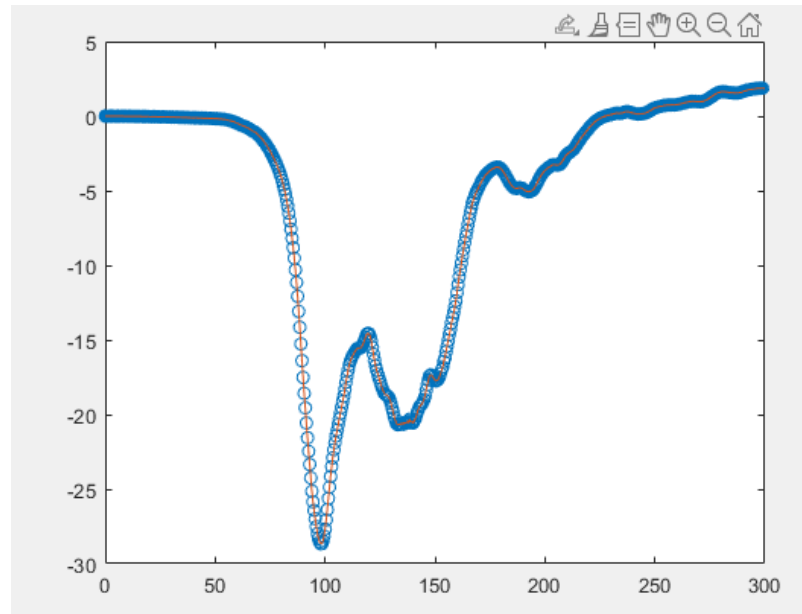
4.5 Spline Interpolation



- An example with more data points
- Head x-acceleration (deceleration event) for a frontal sled test



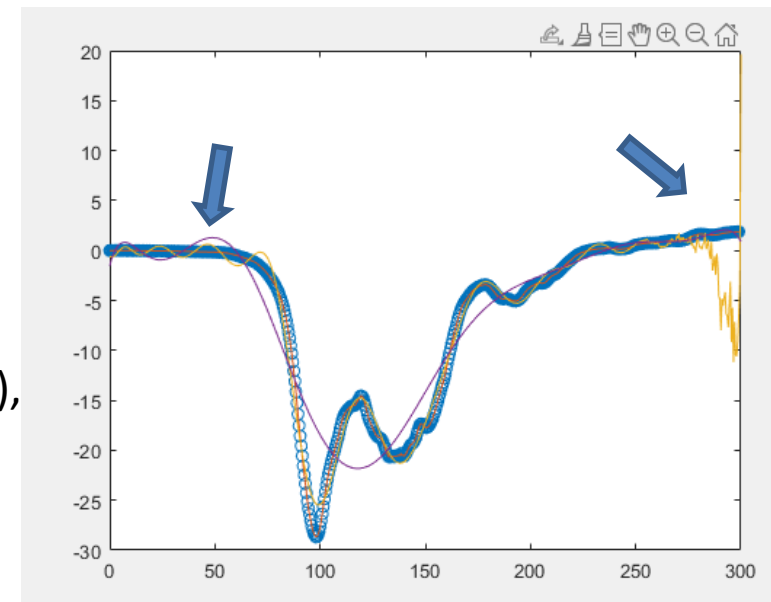
Cubic spline fit to the average curve (red line) with 599 data points
Data points at 0.5 ms intervals



```
1 x = [0 0.5 1 1.5 2 2.5 3 3.5 4 4.5 5  
2 y = [0 -0.001987 -0.002845714 -0.002429857  
3 xx = 0:.5:300;  
4 % cubic spline interpolation in matlab  
5 yy = spline(x,y,xx);  
6 plot(x,y,'o',xx,yy)
```

Polynomial fits start to exhibit oscillations at ~10 terms (red), significant at >50 terms (yellow), doesn't solve ~>100 terms

* problem is ill-conditioned for all poly fits



4.5 Spline Interpolation [Cubic or third-order]

Cubic splines (section 18.6.4)

* See example 18.9 in the text

In this case, we determine the coefficients for 3rd order polynomials corresponding to each interval.

$$s_i(x) = f_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$$

As before, we enforce:

1. Cubics must join at the knots (continuity in magnitude)
2. First derivatives of interior knots must be equal (continuity of first derivative)
3. Second derivatives of interior knots must be equal (continuity of second derivative)
4. To get the required number of equations, we need two more conditions
e.g. can assume the second derivatives at the two end knots are zero

Results in a tri-diagonal set of equations (banded matrix). Example 18.10 in the text book demonstrates this solution.

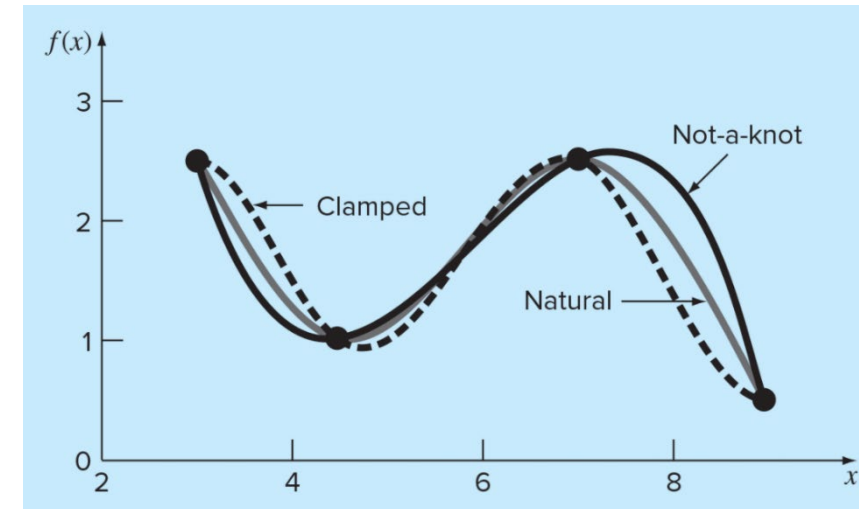
$$\begin{bmatrix} 1 & & & & \\ h_1 & 2(h_1 + h_2) & h_2 & & \\ & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} & \\ & & & 1 & \end{bmatrix} \begin{Bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{n-1} \\ c_n \end{Bmatrix} = \begin{Bmatrix} 0 \\ 3(f[x_3, x_2] - f[x_2, x_1]) \\ \vdots \\ 3(f[x_n, x_{n-1}] - f[x_{n-1}, x_{n-2}]) \\ 0 \end{Bmatrix}$$

4.5 Spline Interpolation [Cubic or third-order, aka Hermite spline]

Cubic splines (section 18.6.5)

Three commonly used end conditions:

1. Natural spline (second derivatives = 0 at end points)
(straightens at ends due to 2nd derivative = 0)
2. Clamped end condition (first derivatives = 0 at the end points)
(levels off to zero slope at ends)
3. Not-a-knot end condition (force continuity of the third derivative at the second and next-to-last knots).
(non-zero 2nd derivatives, exhibits more curvature)



Hermite spline: cubic polynomial with continuity in function values and first derivatives
Commonly used in interpolation of data.